

Analisis de Patrones y Arquetipos - Parcial 2

Asignatura: DSY1106 - Desarrollo Fullstack III
Proyecto: Sanos y Salvos

1. Contexto del problema

El sistema requiere:

- Escalar módulos de negocio de forma independiente (IAM, reportes, matching, etc.).
- Exponer una API unificada y segura para frontend web.
- Tolerar fallas parciales de servicios sin afectar totalmente la experiencia.
- Mantener desarrollo colaborativo por equipos en paralelo.

2. Patrones de diseño implementados

2.1 Patron API Gateway

- **Donde:** módulo gateway.
- **Problema que resuelve:** evita que frontend conozca URLs internas de cada microservicio, centraliza seguridad y enrutamiento.
- **Beneficio:** reduce acoplamiento frontend-backend y simplifica la evolución de endpoints internos.

2.2 Patron BFF (Backend For Frontend)

- **Donde:** módulo bff.
- **Problema que resuelve:** el dashboard necesita datos agregados de varios servicios sin múltiples llamadas desde navegador.
- **Beneficio:** optimiza latencia percibida y encapsula lógica de agregación para frontends.

2.3 Patron Microservicios por dominio

- **Donde:** carpeta `services/` (IAM, reportes, media, foro, etc.).
- **Problema que resuelve:** separa contextos de negocio, permite despliegue/escala independiente y aislamiento de cambios.
- **Beneficio:** mayor mantenibilidad y escalabilidad horizontal por componente.

2.4 Patron Event-Driven (publicador/suscriptor)

- **Donde:** `reports-service` publica evento `report.created`; `audit-service` y `matching-service` consumen.
- **Problema que resuelve:** desacoplar procesos secundarios (auditoría/matching) del flujo transaccional de crear reporte.
- **Beneficio:** mejor resiliencia y menor bloqueo del flujo principal.

2.5 Patron Repository (Spring Data JPA)

- **Donde:** repositorios en cada microservicio (`*Repository`).
- **Problema que resuelve:** separar lógica de acceso a datos de la lógica de negocio.
- **Beneficio:** código más testeable y consistente.

3. Arquetipos y base técnica usada

Para homogeneidad de backend se usa una base común:

- Maven + Spring Boot 3.3.5
- Java 17
- Estructura estándar por módulo:
 - `src/main/java/.../controller`
 - `src/main/java/.../service`
 - `src/main/java/.../repository`
 - `src/main/resources/application.yml`
 - `pom.xml` por servicio

Esta estructura funciona como arquetipo práctico de proyecto para BFF y microservicios.

4. Coherencia arquitectónica (BFF + microservicios)

- gateway expone la frontera pública.
- bff agrega datos de los microservicios para vistas complejas.
- cada microservicio mantiene su propia base `db_*` (database per service).
- RabbitMQ desacopla tareas asíncronas.

Resultado: arquitectura coherente con escalabilidad por servicio y bajo acoplamiento.

5. Eficiencia y mantenibilidad

- dockerización completa con `docker-compose.yml`.
- optimización de build en Dockerfiles usando cache de Maven.
- pruebas unitarias en módulos clave.
- documentación operativa y de troubleshooting en `README.md`.

6. Evidencia en repositorio

- `gateway/` -> patron API Gateway.
- `bff/` -> patron BFF.
- `services/*` -> microservicios por dominio.
- `services/reports-service` + `services/audit-service` + `services/matching-service` -> event-driven con RabbitMQ.

- `src/test` en multiples modulos -> buenas practicas de calidad.